

PEXIP PULSE SDK · DEMONSTRATION

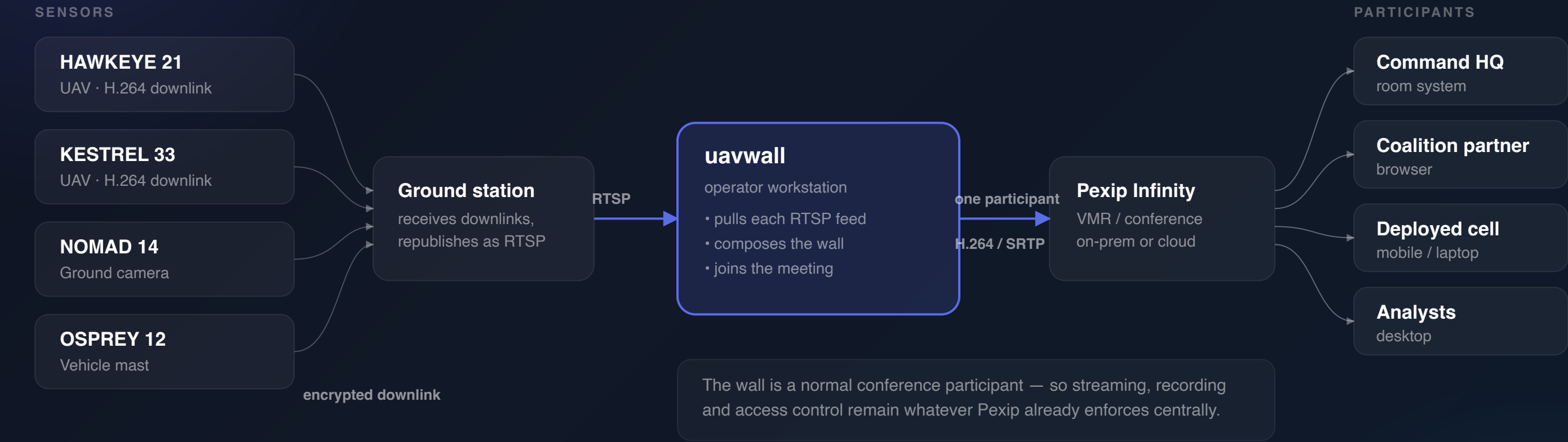
Live UAV feeds, straight into a Pexip meeting

An operator selects from live RTSP downlinks, arranges them on a wall, and that wall is pushed into a Pexip VMR — so every participant, anywhere, sees the same picture in an ordinary video call.

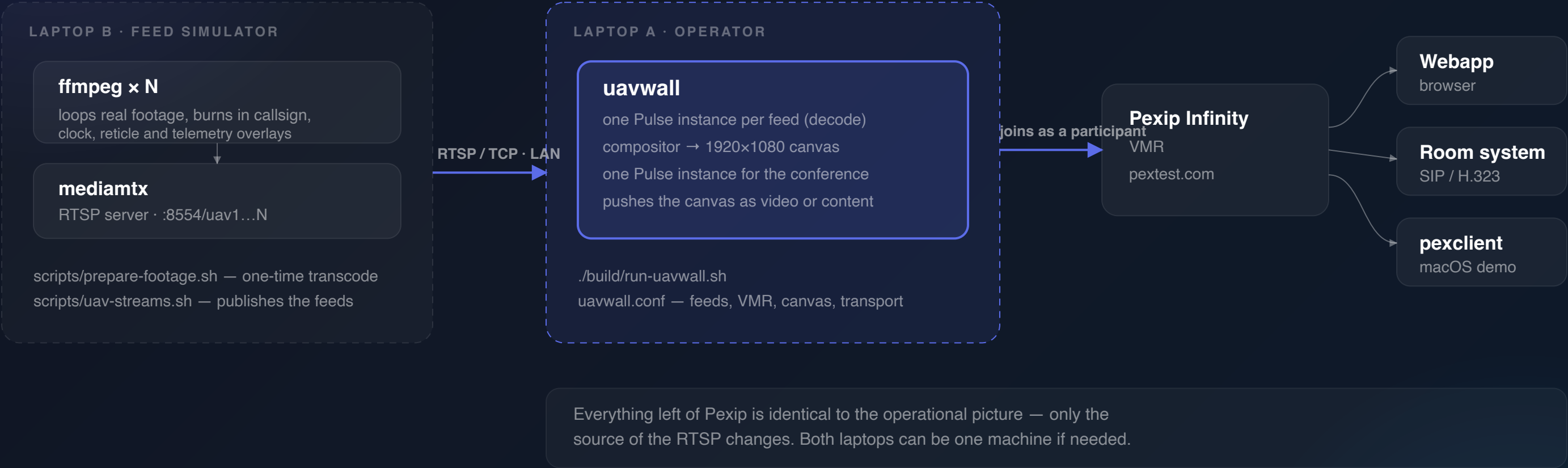
ISR video where the decisions are made

- **Any RTSP source** — UAV downlinks, ground cameras, vehicle feeds. The same protocol every IP camera already speaks.
- **Composed by an operator**, not fixed by configuration — one feed full screen, or nine at a glance, changed live.
- **Delivered as a normal Pexip participant** — no special client, no plugin. Anyone already able to join the meeting can see it.
- **Dialled out, or dialled into** — the wall registers to Infinity, so a conference can call it up on demand.
- **Main video or content** — replace the camera, or send it on the presentation stream alongside the participants.
- **Recorded locally** — any feed at source quality, and the composed wall, straight to MP4.
- **Runs on a small PC** — measured, not estimated; see the sizing slide.
- **Built on the Pulse SDK** — roughly 4,500 lines, one file.

How it fits a real deployment



How the demo is wired



Five actions, no training

1 · Connect

Feeds are listed in the rail with a live thumbnail and a status dot. Connect all, or start the app with feeds already live.

2 · Compose

Click a feed to place it. Presets arrange the wall — 1-up, 2×2, 3×3, PiP — then drag or resize any tile.

3 · Focus

Double-click any feed to punch it full screen; double-click again to return to the previous layout. Nothing is lost.

4 · Send

Enter the VMR and send — or let the conference dial the wall. The canvas becomes the participant's video, or its content stream.

5 · Record

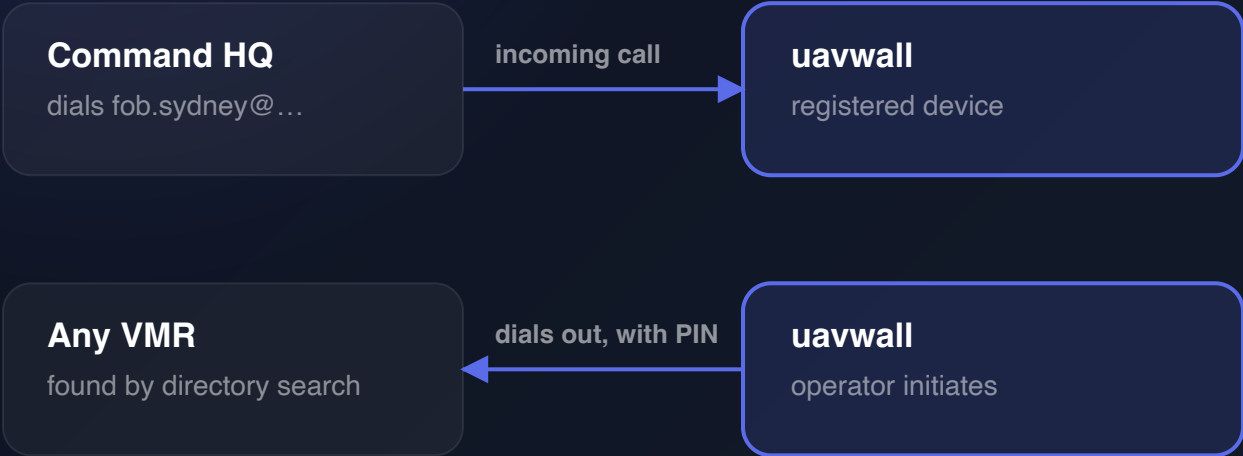
Arm any feed, or the wall itself, at any time. Feeds are captured at **source** quality, independently of what is on the canvas.

Everything is live: rearranging the wall changes what the meeting sees immediately, with no renegotiation and no interruption. Layouts can be saved to three slots and recalled with one click.

What is in the demonstration

- **Any number of RTSP feeds**, named with callsigns, TCP or UDP, connected individually or all at once.
- **Live status per feed** — receiving, stalled, failed — with an inspector giving URL, transport, uptime and last-frame age, and the underlying error surfaced rather than swallowed.
- **Feed-loss alerts** — a source that stops delivering raises an audible and visual alarm, and clears itself when frames resume.
- **Layout presets, free positioning and three saved layouts**, with full-screen punch and restore.
- **Send as main video or as content**, taking the conference floor automatically for content.
- **Registered to Infinity** — the wall can be dialled, searches the directory for VMRs, and answers incoming calls; if it is already in a conference the operator is asked before it leaves.
- **Local recording** — feeds captured by stream copy at source quality for 0.3% of a core, and the composed wall encoded in hardware.
- **Send resolution and frame rate** selectable — the two levers that decide the hardware you need.
- **Live instrumentation** — per-feed rates, process CPU and memory, and the real transmit bitrate, loss and round-trip time from Pulse.
- **Reproducible benchmark** built in, so capacity can be measured on any candidate machine.

Called on demand, and kept afterwards



Registered with a device credential — no per-person sign-in on an unattended wall. A call arriving while it is already engaged is the operator's decision, never a silent drop.

Feeds — captured at source

ffmpeg copies the RTSP stream without decoding it: **0.3% of a core** per recording, and the file holds the original picture, not the downscaled wall tile. Independent of what is on the canvas, or even connected.

The wall — exactly what was sent

The same composed frames that go to the meeting are encoded in hardware to a second file, so the recording is the programme, not a re-creation of it.

Named for the record

HAWKEYE-21_20260729T085346Z.mp4 — callsign and UTC start, into a folder of your choosing.

What it costs to run

FEEDS	CPU	MEMORY	COMPOSITE
1	45%	174 MB	13.8 ms
4	78%	279 MB	16.6 ms
9	114%	470 MB	16.9 ms

720p feeds, 1080p30 output, Apple M4 Pro.
CPU is a percentage of **one** core — 114% is 8% of that machine.

- **A fixed floor of ~45%** — compositing the canvas, almost independent of feed count.
- **~8.6% CPU and 37 MB per extra feed** — decode.
- **720p at 15fps costs 48%** where 1080p at 30fps costs 78% — indistinguishable in a 2×2 grid.

Recommended demo machine

Intel Core Ultra 5/7 or 12th-gen-plus i5/i7 mini PC, 16 GB. Single-core speed is the constraint; Quick Sync offloads decode and encode. Avoid N100-class chips.

Three commands

```
# once – normalise footage to the streaming size
./scripts/prepare-footage.sh UAV_footage

# laptop B – publish the feeds (-a prints a ready-to-paste list)
./scripts/uav-streams.sh -n 4 -d UAV_footage/prepared -a

# laptop A – the operator
./build/run-uavwall.sh
```

No aircraft, no cameras, no internet. Both roles can share one machine; running them apart keeps the measurements honest and puts the feeds on the network, as they would be in service.

Built on the Pexip Pulse SDK

One Pulse instance per feed pulls and decodes RTSP; a further instance joins the conference and accepts the composed frames. The same SDK underlying Pexip's own clients — approximately 4,500 lines of application code.